

## Introducción

La idea de nuestro proyecto es un robot equilibrista. Más específicamente, un robot que puede mantenerse equilibrado y maniobrar utilizando dos ruedas, que además pueda controlarse mediante Bluetooth.

Este robot puede modelarse como un péndulo invertido de dos ruedas: un sistema que consiste de un péndulo montado sobre dos ruedas que pueden girar libremente (fig 1.a). Este sistema es inestable, por lo que es muy utilizado para las pruebas de algoritmos de control. Un ejemplo comercial de este tipo de robots es el Segway (fig 1.b)

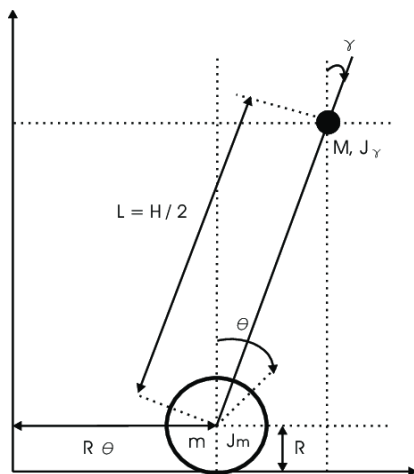
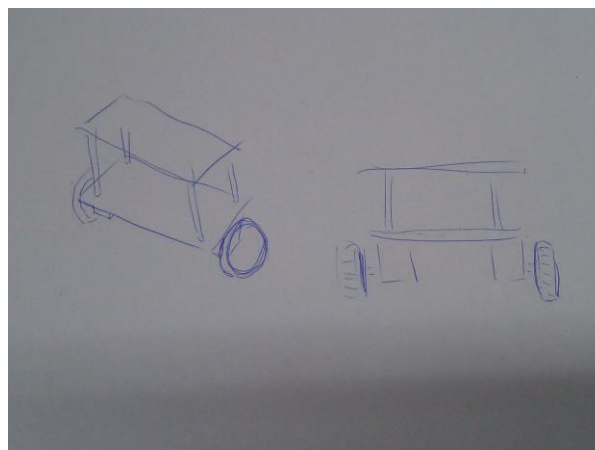


Fig 1.a: Esquema del péndulo invertido

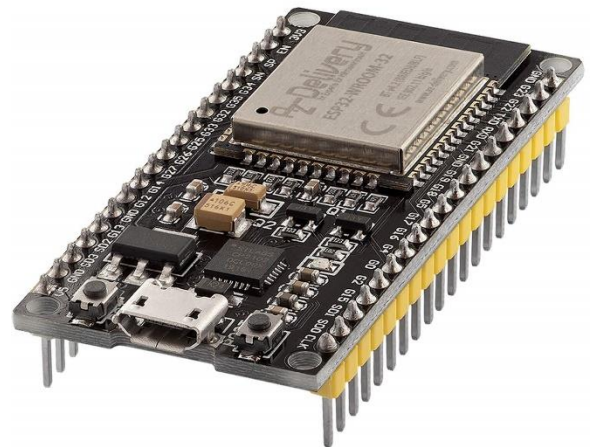


Fig 1.b: Segway

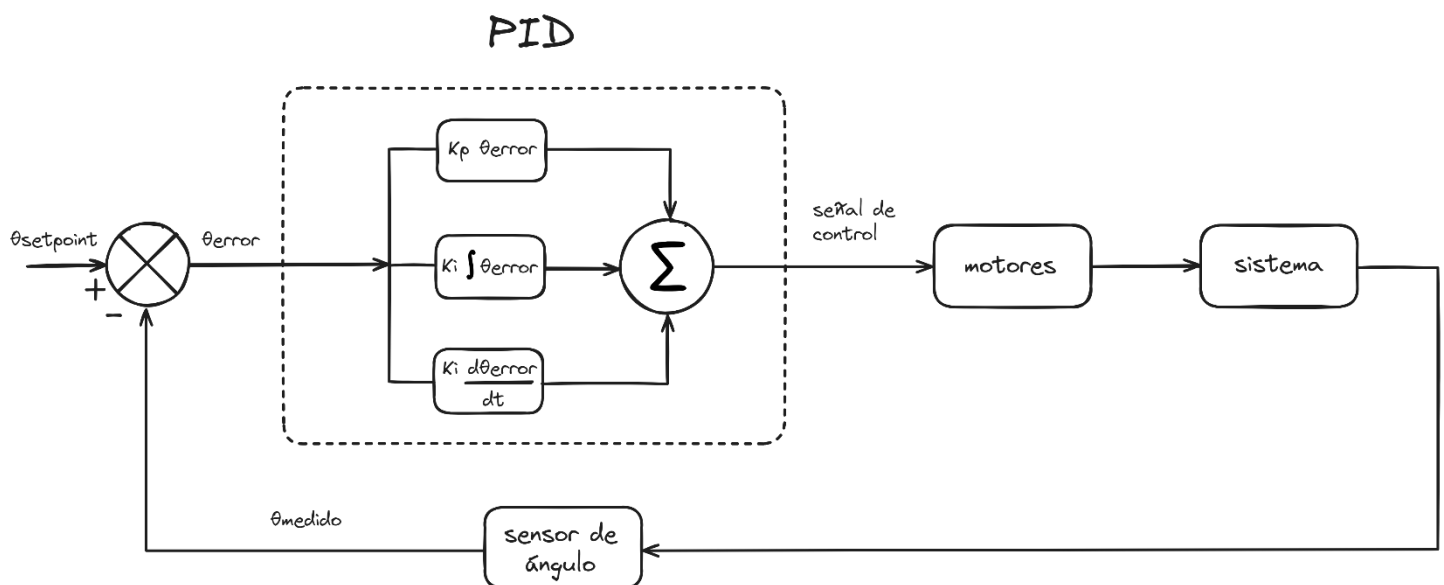
Se puede ver el primer boceto de nuestro proyecto:



La idea es que el robot mantenga el equilibrio controlando únicamente los motores. Claramente necesitamos cierta información del exterior, que obtendremos a partir de sensores. Para leer y procesar la información de los sensores e implementar la comunicación Bluetooth, usaremos un microcontrolador programable: el ESP32. Elegimos este porque ya viene con Bluetooth integrado y tiene librerías que nos facilitan implementar la comunicación, además de que puede programarse en el framework de Arduino con PlatformIO



Investigando, vimos que uno de los algoritmos de control más usados es el control PID, que podemos representar en el siguiente diagrama.



Vamos a considerar que  $\theta$  es el ángulo de inclinación el robot. El setpoint será el ángulo al que queremos llegar, y el error la diferencia entre el setpoint y el ángulo medido.

$$\theta_{\text{setpoint}} - \theta_{\text{medido}} = \theta_{\text{error}}$$

El control PID (proporcional, integrativo, derivativo) es un sistema de control de lazo cerrado en el cual nuestra señal de control es una combinación lineal entre el error, su integral y su derivada, dado por las constantes  $K_p$ ,  $K_i$  y  $K_d$ .

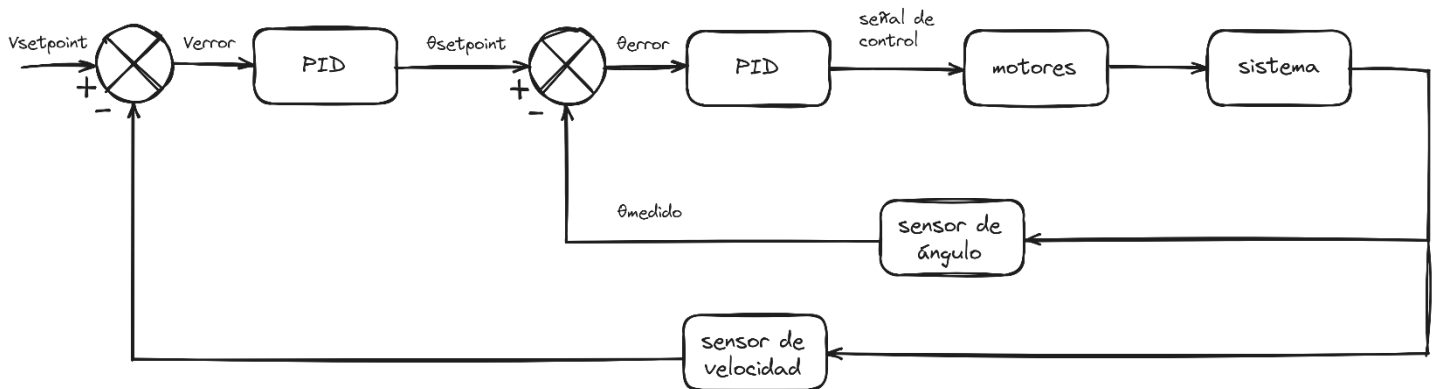
$$\text{señal de control} = K_p \cdot \theta_{\text{error}} + K_i \cdot \int \theta_{\text{error}} dt + K_d \cdot \frac{d\theta_{\text{error}}}{dt}$$

La eficacia de este sistema depende de qué tan bien ajustados estén los tres parámetros  $K_p$ ,  $K_i$  y  $K_d$

Ahora bien, si este sistema de lazo cerrado está bien afinado puede mantener al robot verticalmente de una forma muy estable, pero si queremos maniobrar (modificando el setpoint) el sistema se vuelve muy inestable. (checkear)

Se puede llegar a un sistema mucho más estable si añadimos otro lazo cerrado en cascada, que controle el setpoint de nuestro primer PID en función de la velocidad lineal del robot.

El sistema se representa así:



Con esto ya hemos definido el algoritmo de control. Lo que queda es implementarlo en nuestro proyecto. Todos estos cálculos pueden realizarse fácilmente en el firmware, principalmente necesitamos saber como:

- Controlar los motores
- Medir el ángulo de inclinación
- Sintonizar el primer PID
- Medir la velocidad de las ruedas
- Sintonizar el segundo PID

En las siguientes partes veremos como hacer esto

## Referencias

- [https://es.wikipedia.org/wiki/Controlador\\_PID](https://es.wikipedia.org/wiki/Controlador_PID)
- <https://robotics.stackexchange.com/questions/17964/self-balancing-robot-pid-control-algorithm>
- <https://arxiv.org/ftp/arxiv/papers/2303/2303.12067.pdf>